Training

Document 5 of 6 — Building a GPT from Scratch

This document covers everything that happens between having a model and having a trained model: the data pipeline, the optimizer, the learning rate schedule, and the training loop. Each component has a clear mathematical motivation.

1 Character-Level Tokenization

Before the model can process text, we need to convert characters to integers. TinyShakespeare contains approximately 1,115,394 characters drawn from a vocabulary of 65 unique characters (lowercase and uppercase letters, digits, punctuation, spaces, and newlines).

Our tokenizer (in `src/tokenizer.py`) is the simplest possible: sort the unique characters, assign each an integer index, and store two lookup dictionaries.

```
class CharTokenizer:
    def __init__(self, text: str):
        chars = sorted(set(text))
        self.char_to_idx = {ch: i for i, ch in enumerate(chars)}
        self.idx_to_char = {i: ch for i, ch in enumerate(chars)}

    def encode(self, text: str) -> list[int]:
        return [self.char_to_idx[ch] for ch in text]

    def decode(self, tokens: list[int]) -> str:
        return "".join(self.idx_to_char[t] for t in tokens)
```

Encoding is a character-by-character lookup. Decoding is the reverse. That is the entire tokenizer.

1.1 Comparison with Subword Tokenization (BPE)

Production language models use **Byte Pair Encoding (BPE)** or similar subword tokenization schemes. BPE starts with individual characters (or bytes) and iteratively merges the most frequent pair of adjacent tokens into a new token. After many merges, the vocabulary contains common words as single tokens, rare words as sequences of subword pieces, and characters as a fallback.

Property	Character-level	BPE (subword)
Vocab size	65	30,000 – 50,000
Sequence length for same text	Long (1 char = 1 token)	Short (~4x fewer tokens)
Tokenizer complexity	Two dictionaries	Merge table + encoding algorithm
Information per token	1 character	Roughly 1 word or word piece

BPE produces shorter sequences, which means the model's fixed context window covers more text. It also lets the model reason at word-level granularity rather than character-level. The trade-off is tokenizer complexity and a much larger embedding table.

We use character-level tokenization because it makes the full pipeline transparent. There are no merge rules, no special tokens, no tokenizer artifacts. Every step from raw text to model input is visible.

2 Building the Dataset

The data pipeline is in `src/dataset.py`. The key steps:

1. Load and encode. Download the TinyShakespeare text file, encode all characters to integers, and store the result as a single long tensor.

```
text = download_data()                                          # ~1.1M
    characters
tokenizer = CharTokenizer(text)
data = torch.tensor(tokenizer.encode(text), dtype=torch.long)  #
    shape: (1115394,)
```

2. Train/val split. The first 90% of the data is training, the last 10% is validation. We split sequentially (not randomly) because the data is a continuous text — random splitting would leak context across the boundary.

```
split = int(0.9 * len(data))
train_data = data[:split]    # ~1,003,854 tokens
val_data = data[split:]      # ~111,540 tokens
```

3. Create training examples. Each training example is a contiguous slice of `block_size + 1 = 257` tokens. The input `x` is the first 256 tokens; the target `y` is the same window shifted right by one:

```
def __getitem__(self, idx):
    chunk = self.data[idx : idx + self.block_size + 1]  # 257 tokens
    x = chunk[:-1]    # tokens 0..255 -- the input
    y = chunk[1:]      # tokens 1..256 -- the target
    return x, y
```

Every position in `x` provides a training example. Position 0 must predict position 1 (using only token 0 as context). Position 1 must predict position 2 (using tokens 0 and 1). Position 255 must predict position 256 (using all 256 tokens of context). A single `(x, y)` pair gives us 256 next-token prediction examples.

4. DataLoader. Standard PyTorch DataLoader wraps the dataset:

```
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,           # random order for training
    pin_memory=True,        # faster CPU->GPU transfer
    drop_last=True,         # drop incomplete final batch
)
```

Each batch contains 64 sequences of length 256, giving $64 \times 256 = 16{,}384$ tokens per batch.

3 The Optimizer: AdamW

3.1 Starting from SGD

The simplest optimization algorithm is stochastic gradient descent. Given the gradient g_t of the loss with respect to parameters θ at step t:

$$\theta_{t+1} = \theta_t - \alpha \cdot g_t$$

where α is the learning rate. Two problems: (1) all parameters share the same learning rate, even though different parameters may need very different step sizes; (2) stochastic gradients are noisy (computed on a minibatch, not the full dataset), so each individual update may point in a suboptimal direction.

3.2 Momentum

Maintain a running average of past gradients, called the momentum buffer:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$$\theta_{t+1} = \theta_t - \alpha \cdot m_t$$

This exponential moving average smooths out noise. If gradients consistently point in the same direction, the momentum builds up and the effective step size increases. If gradients oscillate, momentum dampens the oscillation. Typical value: $\beta_1 = 0.9$.

3.3 Adam (Adaptive Moment Estimation)

Adam maintains two running averages — the first moment (mean of gradients) and the second moment (mean of squared gradients):

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

Since $m_0 = 0$ and $v_0 = 0$, the early estimates are biased toward zero. Bias correction compensates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The update rule:

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

The division by $\sqrt{\hat{v}_t}$ gives each parameter its own effective learning rate. Parameters with consistently large gradients (high $\hat{v}_t$) get smaller updates. Parameters with small gradients get larger updates. This per-parameter adaptation is what makes Adam work well across a wide range of architectures without careful learning rate tuning.

Typical values: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

3.4 AdamW: Decoupled Weight Decay

Weight decay is a regularization technique that pushes weights toward zero, preventing the model from relying too heavily on any single parameter. The idea is to add a penalty proportional to the magnitude of the weights.

Adam with L2 regularization (the wrong way): Add $\lambda\theta$ to the gradient before applying the adaptive scaling:

$$g'_t = g_t + \lambda\theta_t$$

Then use g'_t in the Adam update. The problem: the L2 term $\lambda\theta_t$ gets divided by $\sqrt{\hat{v}_t}$ just like the actual gradient. For parameters with large gradients (high $\hat{v}_t$), the regularization effect is weakened. The decay is no longer uniform — it depends on the gradient history, which is not the intended behavior.

AdamW (the right way, Loshchilov & Hutter, 2019): Apply the gradient update and the weight decay as two separate operations:

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \alpha \cdot \lambda \cdot \theta_t$$

The weight decay term $\alpha\lambda\theta_t$ is subtracted directly, bypassing the adaptive scaling. Every parameter is decayed by the same fraction $\alpha\lambda$ per step, regardless of its gradient magnitude.

3.5 Parameter Groups

Not all parameters should be decayed. Weight matrices benefit from decay — it prevents any single connection from becoming too dominant. But biases, LayerNorm parameters, and embeddings should not be pushed toward zero: biases are offsets that need to be whatever value is correct, and LayerNorm parameters control the scale and shift of normalized activations.

Our `configure_optimizer` function in `src/train.py` separates parameters into two groups:

```
for name, param in model.named_parameters():
    if param.dim() < 2:        # biases, LayerNorm weights (1D
        tensors)
        no_decay_params.append(param)
    else:                       # weight matrices (2D tensors)
        decay_params.append(param)

optim_groups = [
    {"params": decay_params, "weight_decay": 0.1},
    {"params": no_decay_params, "weight_decay": 0.0},
]
```

The heuristic is simple: 1D parameters (biases, norm weights) get no decay; 2D parameters (weight matrices) get decay. This covers all the cases correctly for our architecture.

3.6 Our Settings

Parameter	Value
Learning rate	1×10^{-3}
β_1	0.9
β_2	0.99
Weight decay	0.1

Note: we use $\beta_2 = 0.99$ rather than the default 0.999. The smaller value makes the second moment estimate adapt faster to changes in gradient magnitude, which works better for language model training.

4 Learning Rate Schedule: Cosine Annealing with Warmup

The learning rate is not constant during training. It follows a two-phase schedule.

4.1 Phase 1: Linear Warmup (Steps 0 to 100)

The learning rate increases linearly from 0 to the maximum learning rate:

$$\text{lr}(t) = \text{max_lr} \cdot \frac{t}{\text{warmup_iters}}$$

Why warmup is necessary: at the start of training, the model's parameters are random. The loss landscape is chaotic — gradients are large and point in unpredictable directions. Adam's moment estimates m_t and v_t are initialized to zero and have not yet converged to meaningful values. If you start with a large learning rate, the optimizer takes large steps based on unreliable gradient estimates, which can cause the loss to diverge.

Warmup lets the moment estimates stabilize before the learning rate reaches its full value. By step 100, m_t and v_t have accumulated enough gradient history to provide reliable per-parameter scaling.

4.2 Phase 2: Cosine Decay (Steps 100 to 5000)

After warmup, the learning rate decreases following a cosine curve:

$$\text{lr}(t) = \text{min_lr} + \frac{1}{2}(\text{max_lr} - \text{min_lr})\left(1 + \cos\left(\pi \cdot \text{progress}\right)\right)$$

where $\text{progress} = (t - \text{warmup_iters})/(\text{decay_iters} - \text{warmup_iters})$ goes from 0 to 1.

At progress $= 0$, the cosine term is $1 + \cos(0) = 2$, so $\text{lr} = \text{max_lr}$. At progress $= 1$, the cosine term is $1 + \cos(\pi) = 0$, so $\text{lr} = \text{min_lr}$. The decay is smooth and gradual — the learning rate decreases slowly at first, faster in the middle, and slowly again near the end.

Why cosine: compared to step decay (sudden drops at fixed intervals), cosine annealing avoids abrupt jumps that can destabilize training. Compared to linear decay, the cosine curve spends more time near the maximum learning rate early on (when the model is still making large progress) and near the minimum late in training (when fine adjustments matter more).

The original "Attention Is All You Need" paper uses a different schedule: $\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(t^{-0.5}, t \cdot \text{warmup}^{-1.5})$. This is inverse square root decay after warmup. Cosine annealing with warmup has become the modern standard, used in GPT-2, GPT-3, and most subsequent work.

Here is the implementation from `src/train.py`:

```
def get_lr(step: int) -> float:
    # Linear warmup
    if step < WARMUP_ITERS:
        return LEARNING_RATE * step / WARMUP_ITERS
    # After decay period, stay at min
    if step > LR_DECAY_ITERS:
        return MIN_LR
    # Cosine decay
    progress = (step - WARMUP_ITERS) / (LR_DECAY_ITERS -
       WARMUP_ITERS)
    coeff = 0.5 * (1.0 + math.cos(math.pi * progress))
    return MIN_LR + coeff * (LEARNING_RATE - MIN_LR)
```

The learning rate is set manually at each step by writing to the optimizer's parameter groups:

```
lr = get_lr(step)
for param_group in optimizer.param_groups:
    param_group["lr"] = lr
```

5 The Training Loop

The training loop in `src/train.py` follows the standard PyTorch pattern, with mixed precision and gradient clipping. Here is the core of the loop, annotated step by step:

```
for step in range(max_iters):
    # 1. Set learning rate for this step
    lr = get_lr(step)
    for param_group in optimizer.param_groups:
        param_group["lr"] = lr

    # 2. Get a batch from the DataLoader, move to GPU
    x, y = next(train_iter)          # x: (64, 256), y: (64, 256)
    x, y = x.to(device), y.to(device)

    # 3. Forward pass with mixed precision
    with torch.amp.autocast("cuda", dtype=torch.float16,
       enabled=use_amp):
        _, loss = model(x, y)

    # 4. Backward pass with loss scaling
    scaler.scale(loss).backward()

    # 5. Unscale gradients for correct norm computation
    scaler.unscale_(optimizer)

    # 6. Gradient clipping
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    # 7. Optimizer step and scaler update
    scaler.step(optimizer)
    scaler.update()
```

```
    # 8. Clear gradients
    optimizer.zero_grad(set_to_none=True)
```

Let us walk through each piece.

5.1 Mixed Precision Training

Modern GPUs have specialized hardware (tensor cores) that operate much faster on 16-bit floating point than on 32-bit. Mixed precision training exploits this:

- **Forward and backward pass:** Run in float16. This uses roughly half the memory and is faster on tensor cores.
- **Optimizer state and parameter updates:** Kept in float32. The optimizer needs full precision because it computes small corrections (learning rate times gradient divided by moment estimate) that would lose significant digits in float16.

The `torch.amp.autocast` context manager automatically casts operations to float16 where it is safe and leaves precision-sensitive operations (like softmax, loss computation) in float32.

The GradScaler: Float16 has a limited range — the smallest representable positive number is about 6×10^{-8}. Gradients for some parameters can be smaller than this, which means they round to zero (underflow). The GradScaler multiplies the loss by a large factor before `backward()`, which scales all gradients up into the representable range. Before the optimizer step, `scaler.unscale_()` divides the gradients back to their true values. The scale factor is adjusted dynamically: if gradients contain inf/nan (overflow), the scaler reduces the factor and skips the optimizer step.

5.2 Gradient Clipping

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

This computes the global norm of all gradients:

$$\|g\| = \sqrt{\sum_i \|g_i\|^2}$$

where the sum is over all parameter tensors. If $\|g\| > \text{max_norm}$, every gradient is scaled down by $\text{max_norm}/\|g\|$ so that the global norm becomes exactly `max_norm`.

Why this matters: occasionally, a batch produces anomalously large gradients (an "exploding gradient"). Without clipping, a single bad batch can make a destructive parameter update that ruins training progress. Gradient clipping bounds the worst-case step size while leaving normal-sized gradients unchanged.

Note: `scaler.unscale_()` must be called before gradient clipping. Otherwise you would be clipping the scaled gradients, and the effective max norm would vary with the current scale factor.

5.3 Clearing Gradients

```
optimizer.zero_grad(set_to_none=True)
```

PyTorch accumulates gradients by default — calling `.backward()` adds to the existing `.grad` attribute. We need to clear gradients before each step. The `set_to_none=True` argument sets `.grad = None` instead of filling with zeros. This is slightly faster (avoids a memset) and uses no memory until the next backward pass populates the gradients.

5.4 Periodic Evaluation

Every 250 steps, the training loop evaluates the model on both training and validation data by averaging the loss over 200 batches:

```
@torch.no_grad()
def estimate_loss(model, train_loader, val_loader, device,
    eval_iters=200):
    model.eval()
    # ... average loss over eval_iters batches for each split ...
    model.train()
    return {"train": train_loss, "val": val_loss}
```

`model.eval()` disables dropout (all tokens are kept, but activations are not scaled — PyTorch handles this automatically). `@torch.no_grad()` disables gradient tracking, saving memory during evaluation. After evaluation, `model.train()` re-enables dropout.

5.5 Checkpointing

When the validation loss improves, the model state is saved:

```
checkpoint = {
    "model": model.state_dict(),
    "config": config,
    "step": step,
    "val_loss": best_val_loss,
    "vocab": tokenizer.char_to_idx,
}
torch.save(checkpoint, CHECKPOINT_DIR / "best.pt")
```

The checkpoint includes everything needed to reconstruct the model: the weights, the architecture config, and the vocabulary mapping. This is what `src/generate.py` loads to produce text.

6 Hyperparameters Summary

Parameter	Value	Rationale
`batch_size`	64	$64 \times 256 = 16{,}384$ tokens per batch. Fits comfortably in 10GB VRAM.
`max_iters`	5000	$5000 \times 16{,}384 \approx 82$M tokens seen. Multiple passes over the 1M-token dataset.
`learning_rate`	1×10^{-3}	Relatively high — small models tolerate larger learning rates.
`min_lr`	1×10^{-4}	10x below max. The model fine-tunes in the late phase.
`warmup_iters`	100	2% of training. Enough for moment estimates to stabilize.
`weight_decay`	0.1	Standard for transformer training. Prevents weight growth.
`grad_clip`	1.0	Standard value. Rarely triggers but prevents catastrophic steps.
`beta1`	0.9	Standard momentum coefficient.
`beta2`	0.99	Slightly faster adaptation than default 0.999.

Expected results: Validation loss of approximately 1.47 after 5000 iterations. Training time approximately 3–5 minutes on an RTX 3080. The model will produce text that looks like plausible Shakespeare formatting with quasi-Elizabethan words, though it will not be semantically coherent (it is a 10.7M parameter character-level model trained on 1MB of data).

7 Label Smoothing (Brief Note)

The original "Attention Is All You Need" paper uses label smoothing with $\epsilon = 0.1$. Instead of the standard one-hot target distribution:

$$\text{target} = [0, 0, \ldots, 1, \ldots, 0]$$

label smoothing replaces it with:

$$\text{target}_i = \begin{cases} 1 - \epsilon + \epsilon/K & \text{if } i = \text{correct class} \\ \epsilon/K & \text{otherwise} \end{cases}$$

where K is the number of classes (vocabulary size). For our model with $K = 65$ and $\epsilon = 0.1$: the correct token gets probability $0.9 + 0.1/65 \approx 0.9015$, and each incorrect token gets probability $0.1/65 \approx 0.00154$.

This prevents the model from becoming overconfident. Without label smoothing, the model is incentivized to push logits for the correct token toward $+\infty$ and all others toward $-\infty$. Label smoothing caps the benefit: once the model assigns ~90% probability to the correct token, there is no further gradient signal pushing it higher. This can improve generalization by preventing the model from memorizing training examples too precisely.

When label smoothing is used, the loss is typically computed as KL divergence between the smoothed target distribution and the model's output distribution, rather than standard cross-entropy.

We do not use label smoothing in our BabyGPT. At this scale (small model, small dataset), the benefit is minimal, and standard cross-entropy keeps the code simpler.

Next: Document 06 — Generation and Sampling covers autoregressive generation, temperature, top-k sampling, and what to expect from our trained model.